EASE-Doc - A Framework for Cost-Driven NoSQL Schema Selection

Morgan O. M. Siqueira[1*], Dimas Cassimiro Nascimento[2] and Bruno Nogueira[3]

[1*]Escola Politécnica de Pernambuco – Universidade de Pernambuco (UPE), Recife, PE, Brasil.
[2]Universidade Federal do Agreste de Pernambuco (UFAPE), Garanhuns, PE, Brasil.
[3]Universidade Federal de Alagoas (UFAL), Maceió, AL, Brasil.

*Corresponding author(s). E-mail(s): moms@ecomp.poli.br;
Contributing authors: dimas.cassimiro@ufape.edu.br; bruno@ic.ufal.br;

Abstract

Document-oriented NoSQL databases provide structural flexibility but make schema design highly dependent on application workloads. Decisions about referencing, embedding, and data duplication directly affect read performance and the cost of maintaining redundant data, and teams often make these choices empirically. This paper presents EASE-Doc, an automated framework that extracts, evaluates, and selects document-oriented schemas through workload-aware, cost-driven refactoring. EASE-Doc analyzes application source code to recover the base schema, derive read and write access patterns, and identify opportunities for selective duplication. Candidate schemas are evaluated using a cost model that combines read and write costs according to the workload profile, while Tabu Search explores the resulting design space. We evaluated the framework on a MongoDB Marketplace application using Yahoo! Cloud Serving Benchmark (YCSB) under different read and write workload profiles. The results show that the cost model preserves the experimentally observed performance ordering. At the same time, Tabu Search identified the same minimum-cost schemas as exhaustive search while evaluating less than 1.3% of the candidate space, substantially reducing the search effort without compromising solution quality. The selected schemas reduced weighted average latency by up to 24.7% compared with the base schema, while retaining the original structure when writes predominated. Overall, EASE-Doc effectively adapts schema refactoring to different workload profiles while substantially reducing the search effort.

Keywords: NoSQL, Document-Oriented Databases, Schema Optimization, Cost Model

1 Introduction

NoSQL databases have become an important alternative to relational systems for applications requiring scalability, availability, and structural flexibility [1, 2]. Among them, document-oriented databases such as MongoDB allow semi-structured data to be organized without requiring a rigid schema declaration. This flexibility facilitates schema evolution, but also increases the number of possible representations for the same domain [3, 4].

In document-oriented databases, structural decisions directly affect application performance. Related data may be represented through references, embedding, or selective duplication, and the effectiveness of each alternative depends on how the application accesses and modifies the data [5–7]. Structures that reduce navigation across collections may improve read performance, whereas duplicated data introduces additional storage and maintenance costs when updates must be propagated. Consequently, there is no universally optimal document schema: its suitability depends on the application workload.

Despite this dependency, schema design and refactoring are still frequently guided by generic recommendations or trial-and-error practices [7, 8]. Existing research addresses several aspects of this problem, including schema inference, workload-driven design, and cost-based evaluation. However, these capabilities are typically addressed in isolation rather than as part of an integrated design process.

Workload-aware approaches often require explicit conceptual models and access patterns [5, 6]. At the same time, schema inference methods generally focus on recovering structural information and do not automatically use it to select refactorings based on operational cost [9–11]. In addition, cost-based approaches for document stores have primarily focused on query performance and read-oriented optimization [7, 12], with limited treatment of the maintenance overhead introduced by selectively duplicated data. As a result, existing solutions provide only partial support for automatically deriving, evaluating, and selecting document schemas while jointly considering read and write costs.

To address these limitations, this paper presents EASE-Doc (*Extraction, Assessment, and Selection of Schemas in Document-Oriented NoSQL Databases*), a workload-aware framework that integrates schema extraction, cost-based evaluation, and automatic schema selection. EASE-Doc starts from the application source code and uses a model-driven reverse-engineering process [11] to recover the base schema, access dependencies, and candidate opportunities for selective field duplication. The extracted artifacts are interpreted as read and write patterns, which are combined with workload and data-volume information to evaluate alternative schemas. The evaluation extends a physical cost model for document stores [12] by incorporating the cost of write propagation, while a Tabu Search strategy explores combinations of candidate duplications and selects the schema with the lowest estimated cost.

The main contributions of this work are:

- an integrated process that connects code-based schema extraction with workload-aware evaluation and automatic schema selection;
- the derivation of explicit read and write patterns from the artifacts produced by the reverse-engineering process, enabling their direct use by the cost model;
- an extension of a physical cost model for document stores to jointly account for read costs and the propagation of writes over selectively duplicated data; and
- a cost-driven search strategy that explores combinations of selective duplications without requiring exhaustive enumeration of the schema space.
- an experimental evaluation validating cost-based ranking accuracy, search efficiency against exhaustive search, and runtime performance improvements across different read/write workloads.

The remainder of this paper is organized as follows. Section 2 presents the background and related work. Section 3 describes the EASE-Doc framework. Section 4 presents the experimental methodology, and Section 5 discusses the results. Finally, Section 6 summarizes the findings, limitations, and directions for future work.

2 Related work

This section outlines some key background notions and summarizes the primary methods for designing, deriving, and optimizing document-oriented NoSQL schemas.

2.1 Document-oriented schema design

Document-oriented databases organize semi-structured data into collections of documents and provide greater structural flexibility than traditional relational systems [1, 2]. Although these systems are often described as *schemaless*, the absence of an explicit schema does not imply the absence of data structure. Instead, part of the schema definition is shifted to the application and to the way documents are organized and accessed [3, 11].

This flexibility enables related data to be represented through different combinations of references and embedded structures. A reference keeps related entities in separate collections and connects them through identifiers, whereas embedding stores related information within the same document. These alternatives have different effects on storage, query performance, redundancy, and maintainability [3, 4]. Consequently, different document structures representing the same domain may exhibit substantially different behavior under the same set of operations.

Data duplication provides an additional design alternative. Instead of embedding an entire related entity, selected attributes may be replicated in documents in which they are frequently required. This strategy is related to the *Extended Reference Pattern*, which recommends copying frequently accessed fields from referenced documents to reduce additional accesses [13, 14]. Although duplication can improve read performance, it introduces redundancy and requires additional maintenance when the original values are modified.

Therefore, document-oriented schema design is inherently dependent on the application *workload*. Structures that reduce navigation among collections can favor read-intensive workloads, whereas designs with less redundancy may be preferable when updates are frequent [5, 6]. This dependence makes generic modeling rules insufficient for determining a universally optimal document schema and motivates methods that explicitly incorporate data access patterns into the design process [7, 8].

2.2 Schema extraction and representation

The flexibility provided by document stores also makes schema understanding more difficult, particularly when the database does not maintain an explicit schema declaration. Nevertheless, structural information remains implicit in persisted documents and in the application code responsible for accessing them [10, 11]. Making this structure explicit is useful for tasks such as data integration, schema evolution, query formulation, and database refactoring.

Several approaches infer schemas directly from persisted documents. Klettke et al. [9] analyze JSON collections to identify structural variations and outliers, whereas Frozza et al. [10] derive a global JSON Schema from JSON and Extended JSON document collections. Other proposals extend schema inference to richer semantic relationships or heterogeneous environments [15, 16]. These methods demonstrate that useful structural information can be recovered from stored data; on the other hand, approaches centered on database instances cannot necessarily recover information that exists only in application logic, such as navigation paths and implicit dependencies among successive database operations.

A complementary line of research adopts metamodels to provide technology-independent schema representations. Fernández-Candel et al. [2] proposed U-Schema, a unified logical metamodel capable of representing relational databases and the main NoSQL paradigms. For document stores, U-Schema explicitly represents attributes, aggregates, references, and structural variations, providing a common abstraction on which model-driven database operations can be performed.

More recently, Fernández-Candel et al. [11] also proposed a reverse-engineering process that extracts NoSQL schemas from application source code using static analysis and Model-Driven Engineering (MDE) [17]. The process progressively transforms application code into intermediate models, including a control-flow representation and the *Database Operations and Structure* (DOS) model, which captures database operations together with the structures manipulated by the application. The resulting logical schema is represented in U-Schema. In addition, dependencies identified between database accesses can reveal opportunities for schema refactoring through field duplication.

EASE-Doc adopts this code-based reverse-engineering process as the foundation of its extraction stage. However, while the original approach can identify potential refactoring opportunities, EASE-Doc interprets the extracted artifacts to derive explicit read and write access patterns and subsequently evaluates which candidate duplications should be adopted according to their estimated cost.

2.3 Workload-driven schema optimization

Researchers have investigated workload-aware design in several NoSQL contexts. NoSE [18] proposes a cost-based schema design method for extensible record stores, such as Cassandra. Given a conceptual data model and a workload composed of queries and updates, the method generates candidate column families and uses Binary Integer Programming to select a schema with low estimated execution cost. Although NoSE demonstrates the benefit of jointly considering query and update operations, its schema representation and optimization process target wide-column stores rather than document-specific structures.

For document databases, de Lima and Mello [5] propose a logical design process that transforms an EER conceptual schema into a document-oriented schema. The method considers estimated data volumes and application operations to determine how conceptual elements should be mapped to document structures. The approach shows that workload information can guide decisions between alternative representations, but requires the conceptual model and workload to be explicitly provided by the designer.

Chen et al. [6] propose a more general workload-driven method for aggregate-oriented NoSQL databases. Their approach introduces the *Query Path Graph* (QPG) as an intermediate logical representation and maps conceptual structures and access patterns to key-value, wide-column, and document schemas. A cost model is used to optimize the resulting structures by considering query and update performance as well as storage and consistency trade-offs. Nevertheless, the method starts from an explicitly defined conceptual model and predefined query and update patterns rather than recovering these elements from an existing application.

Other studies focus more specifically on evaluating document structures. Gómez et al. [4] generate alternative document structures from an initial UML model and use structural metrics to compare their complexity, redundancy, and nesting characteristics. Vera-Olivera et al. [19] experimentally compare referenced and embedded document structures and show that their relative performance varies according to the access characteristics. These approaches reinforce the importance of schema structure, although they do not provide an integrated process for extracting application workloads and automatically selecting refactorings.

A complementary contribution is the cost model proposed by Hewasinghage et al. [12]. Instead of relying only on structural heuristics, the model estimates the relative cost of random-access queries by considering physical factors such as document and index sizes, memory allocation, cache behavior, and disk accesses. The model was incorporated into DocDesign [20], which allows users to compare alternative document designs under a given query workload without materializing every alternative.

DocDesign was subsequently extended into an automated multicriteria design approach [7]. The method formalizes transformations over document schemas and applies local search to identify near-optimal structures according to objectives such as query performance, storage space, document depth, and structural heterogeneity. This substantially reduces the dependence on manual trial-and-error design. Yet, the optimization is driven by an explicitly provided query workload and primarily evaluates read-oriented design objectives; the propagation cost caused by updates to selectively duplicated attributes is not the central optimization criterion.

Table 1 summarizes the approaches most closely related to EASE-Doc according to their support for document stores, automatic schema extraction, workload awareness, read/write evaluation, and automatic schema selection.

Overall, the literature provides complementary support for schema inference [9–11], workload-driven design [5, 6, 18], and cost-based evaluation [7, 12, 20]. In contrast, these capabilities are generally addressed separately or depend on manually specified schemas and access patterns. EASE-Doc combines them in a single workflow by extracting the base schema and access patterns from application code, evaluating selective duplication alternatives under both read and write costs, and automatically selecting the lowest-cost schema.

Table 1: Comparison of EASE-Doc with closely related approaches.

Approach	Document store	Automatic extraction	Workload aware	Read and write	Automatic selection
NoSE [18]	×	×	✓	✓	✓
De Lima et al. [5]	✓	×	✓	✓	∼
Chen et al. [6]	∼	×	✓	✓	✓
Hewasinghage et al. [12]	✓	×	✓	×	×
DocDesign 2.0 [7]	✓	×	✓	×	✓
Fernández-Candel et al. [11]	✓	✓	∼	×	×
EASE-Doc	✓	✓	✓	✓	✓

✓ supported; × not supported; ∼ partially supported.

3 Framework

This section presents the EASE-Doc *framework*, designed to extract, assess, and select optimized schemas for document-oriented NoSQL databases. To clarify the process, we use a unified *Marketplace* example implemented in JavaScript and using MongoDB as the document database management system. We use this example throughout the section to show how a schema is extracted from the source code, evaluated through a cost model, and used to select refactoring alternatives. We use the *Marketplace* scenario throughout this section to illustrate each module's operation and its integration within the framework.

As illustrated in Figure 1, the EASE-Doc framework takes application source code and a configuration file as inputs to calibrate a cost model that measures the theoretical cost of document-oriented database schemas under a given *workload* in an integrated manner. The process is organized into three interconnected modules: the Extractor, which analyzes the source code and generates the intermediate DOS and U-Schema models to represent the logical structure, access patterns, and refactoring opportunities; the Selector, which explores the search space by generating candidate schemas

with different duplication plans; and the Evaluator, which estimates the physical and operational cost of each candidate based on the workload characteristics and the configuration file. At the end of the process, the framework selects and recommends the structural alternative with the lowest estimated cost for the application.

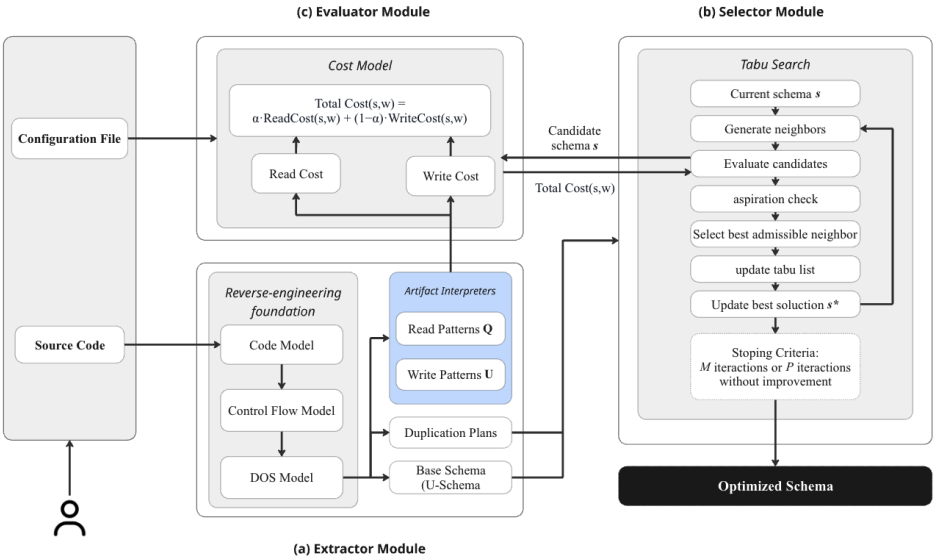


Fig. 1: Overview of the proposed solution.

3.1 Source Code

EASE-Doc receives as input the source code files from the application's server-side layer that handle query, insertion, update, and deletion operations. From these files, the *framework* identifies the accessed collections, the attributes used, and the implicit relationships between entities, and uses this evidence to reconstruct the logical schema and characterize the application's access patterns. In the example shown in Figure 2, a query to the `products` collection, performed through the `findOne` method using the `name` attribute, retrieves a product whose `sellerId` and `categoryId` fields are then used to access the `sellers` and `categories` collections.

This sequence of accesses makes it possible to infer the relationships between the product and its seller and category, while also identifying attributes consumed by the application, such as `stock`, `price`, `storeName`, `rating`, and `name`. The code also contains an `updateOne` operation that locates the product using the `name` attribute and updates the `price` field with the `$set` operator. Therefore, when the source code is submitted to the tool, EASE-Doc identifies both read patterns involving multiple collections and write patterns. It then uses this information to reconstruct the logical schema and characterize the workload.

```
const db = client.db('marketplace');

const product = await db.collection('products')
  .findOne({ name: 'Smartphone X' });

const [seller, category] = await Promise.all([
  db.collection('sellers')
    .findOne({ _id: product.sellerId }),
  db.collection('categories')
    .findOne({ _id: product.categoryId })
]);

if (product.stock > 0) {
  console.log(product.name, product.price);
  console.log(seller.storeName, seller.rating);
  console.log(category.name);
}

await db.collection('products').updateOne(
  { name: 'Smartphone X' },
  { $set: { price: 2499.90 } }
);
```

Fig. 2: Simplified example of application source code.

3.2 Extractor Module

The Extractor module follows the reverse-engineering approach proposed by Fernández-Candel et al. (2026) [11], which analyzes application source code to recover structural and operational information related to persistence in NoSQL databases. This approach uses a sequence of model-driven transformations to progressively convert source code into intermediate representations that make the application structure and its database interactions explicit. In EASE-Doc, this process forms the foundation for recovering the existing schema and the information needed to identify data-duplication alternatives.

Figure 3 provides an overview of the Extractor module and distinguishes the two approaches that compose it. On the left, the reverse-engineering process proposed by Fernández-Candel et al. (2026) [11] transforms the source code into intermediate models, from which the base schema in *U-Schema* and the duplication plans are derived. On the right, the interpreters proposed in EASE-Doc process these artifacts to generate read and write patterns, which the Evaluator module then uses.

3.2.1 Reverse-engineering foundation

The first part of the Extractor module follows Fernández-Candel et al.'s (2026) reverse-engineering approach [11]. The process first generates a *Code Model*, which captures persistence-related elements such as method calls, variables, arguments, property accesses, and code blocks. It then transforms this representation into a *Control Flow Model*, which adds execution order and data dependencies among these elements.

In the *Marketplace* example of Figure 2, the *Code Model* captures the calls to `findOne`, the `product` variable, and accesses to `sellerId` and `categoryId`. The corresponding *Control Flow Model* links these elements by showing that the query to

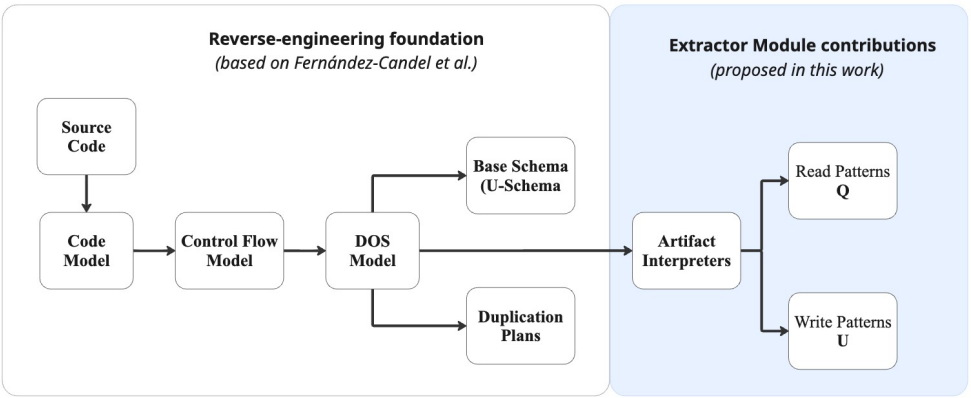


Fig. 3: Workflow of the Extractor module.

`products` precedes and provides the values required by the subsequent accesses to `sellers` and `categories`. By combining structural information from the *Code Model* with execution dependencies from the *Control Flow Model*, EASE-Doc reconstructs access paths and infers implicit relationships among collections.

The control-flow analysis results in the DOS (*Database Operations and Structure*) model, which consolidates, in a single representation, the structural and operational information identified in the source code [11]. Structurally, DOS represents the accessed collections, their fields, and the relationships inferred among them. Fields the application directly manipulates are represented as attributes, whereas fields used to retrieve documents from other collections are identified as references. Operationally, the model records read and write operations, as well as dependencies among successive queries. According to Fernández-Candel et al. (2026) [11], DOS acts as a bridge between the behavior observed in the program and the reconstruction of an explicit schema.

As illustrated in Figure 4, the DOS model can be understood as an unordered list of elements that includes both storage structures and database operations identified in the source code. In the *Marketplace* scenario, this representation includes the `products` collection, with the attributes `name`, `price`, and `stock`, as well as the references `sellerId` and `categoryId`. The model also records read operations that start from `products` and access `sellers` and `categories`, as well as the update operation on `products`. In this way, information originally scattered throughout the application logic is consolidated into a single representation of the structures, relationships, and database access operations.

From the structural information represented in the DOS, the approach of Fernández-Candel et al. [11] derives a logical schema representation using U-Schema [2]. In this mapping, collections are represented as entity types, while fields identified in the code are converted into attributes, references, or aggregations. This resulting representation forms the base schema EASE-Doc uses before any refactoring.

- **Structure**

  `products`
  - `name`: Attribute
  - `price`: Attribute
  - `stock`: Attribute
  - `sellerId`: Reference
  - `categoryId`: Reference

  `sellers`
  - `_id`: Attribute
  - `storeName`: Attribute
  - `rating`: Attribute

  `categories`
  - `_id`: Attribute
  - `name`: Attribute

- **Database Operations**
  - `Read: findOne()` on `products`
    → `JoinQuery: findOne()` on `sellers`
    → `JoinQuery: findOne()` on `categories`;
  - `Update: updateOne()` on `products`.

Fig. 4: Simplified representation of the DOS model.

The same analysis process also identifies refactoring opportunities associated with dependent queries across collections. When a retrieved document provides a reference that is later used to access another collection, the attributes consumed from the referenced entity can be candidates for duplication in the source entity. This strategy follows the *Extended Reference Pattern*, which replicates only a subset of frequently accessed attributes while preserving the original reference [14].

Figure 5 illustrates the duplication opportunities identified in the *Marketplace* scenario. In the first case, accessing `Seller` from `Product` indicates the possibility of duplicating the `storeName` and `rating` attributes into `Product`; similarly, the `name` attribute from `Category` can be incorporated into the same entity. These opportunities form the set of duplication plans that EASE-Doc subsequently uses. The Extractor module does not implement these modifications; instead, it treats them as candidate structural alternatives whose activation is determined during selection.

```
Duplication:
    D1. Copy: Seller[storeName, rating] to: Product
    D2. Copy: Category[name] to: Product
```

Fig. 5: Example of duplication opportunities identified in the *Marketplace* scenario.

3.2.2 Artifact Interpreters

Although the previous artifacts provide the base schema and the refactoring opportunities, their original representation is not directly structured in the format required by the EASE-Doc cost model. To enable this integration, this work introduces artifact interpreters that process the DOS model, represented in XMI (*XML Metadata Interchange*), and convert the extracted operations into normalized read and write patterns.

These artifact interpreters are the main contribution of the Extractor module and fall into two groups: *read patterns* and *write patterns*. *Read patterns* describe read operations by indicating the collections accessed and the relationships each query traverses. In contrast, the *write patterns* represent insertion, update, and deletion operations, including duplicated data that must remain consistent. Based on this information, EASE-Doc estimates the read and write costs of each candidate schema, providing the inputs the Evaluator and Selector modules require.

Algorithm 1 describes the process used to derive *read patterns* from the DOS model extracted from the source code. In lines 1 and 2, the algorithm parses the DOS artifact in XML format. It identifies the section of the model containing database operations extracted from the source code, such as reads, insertions, updates, and deletions. Lines 3–5 then extract three sets of information required to interpret the reads: the database containers, which represent the available collections; the query blocks identified in the source code; and the read operations associated with these blocks. In lines 7–10, each read operation is processed to obtain the reference to the accessed container and resolve it to the corresponding collection name. Once the collection is identified, it is grouped in map M, using the query block in which the read occurs as the key.

Algorithm 1: Extraction of *read patterns*

Input : DOS artifact DOS, query prefix p
Output: set of read patterns R

1 $root \leftarrow \textsc{ParseXML}(DOS)$;
2 $dbo \leftarrow \textsc{FindDatabaseOperationsSchema}(root)$;
3 $C \leftarrow \textsc{ParseContainerNames}(dbo)$;
4 $B \leftarrow \textsc{ParseQueryBlocks}(root)$;
5 $O \leftarrow \textsc{ParseReadOperations}(dbo)$;
6 $M \leftarrow \emptyset$
7 **foreach** $(b, c_{ref}) \in O$ **do**
8 $c \leftarrow \textsc{ResolveContainerName}(c_{ref}, C)$;
9 **if** $c \neq \varnothing$ **then**
10 $M[b] \leftarrow M[b] \cup \{c\}$;
11 $R \leftarrow \emptyset$;
12 $i \leftarrow 1$;
13 **foreach** $(b, f) \in B$ **do**
14 $L \leftarrow M[b]$;
15 **if** $L = \emptyset$ **then**
16 **continue**;
17 $Q \leftarrow \emptyset$
18 **foreach** $c \in L$ **do**
19 $Q[c] \leftarrow Q[c] + 1$;
20 $R[p_i] \leftarrow Q$;
21 $i \leftarrow i + 1$;
22 **return** R;

This process can be illustrated using Figure 2, in which data access begins with the `products` collection and, based on the references contained in the retrieved document, proceeds to the `sellers` and `categories` collections. In lines 13–21, this transformation generalizes to all query blocks extracted from the code: the algorithm disregards blocks without read operations, counts how many times each collection is accessed, and preserves the order of these accesses. We then associate each resulting pattern with an identifier Q_i. Consequently, each *read pattern* is represented as a sequence of pairs (c, n), where c denotes the accessed collection and n represents the number of accesses performed. Finally, line 22 returns the set R, which is subsequently used by the Evaluator module to estimate the read cost of candidate schemas. For the example in Figure 2, the resulting pattern is:

$$Q8 = \{(\texttt{products}, 1), (\texttt{sellers}, 1), (\texttt{categories}, 1)\}.$$

Algorithm 2 describes the process used to derive *write patterns* from the DOS artifact. First, in lines 1 and 2, the DOS artifact is parsed as XML, and the section representing database operations is located. Then, in line 3, the model collects the identified database access operations, including creation, update, and deletion operations, along with any auxiliary reads associated with them.

Algorithm 2: Extraction of *write patterns*

Input : DOS artifact DOS
Output: set of *write patterns* W

```
1  r ← ParseXML(DOS);
2  dbo ← FindDatabaseOperationsSchema(r);
3  O ← CollectDatabaseOperations(dbo);
4  W ← ∅;
5  counts ← {C : 0, U : 0, D : 0};
6  foreach op ∈ O do
7      if not IsWriteOperation(op) then
8          continue;
9      kind ← GetWriteKind(op);
10     prefix ← TypePrefix(kind);
11     if prefix = ∅ then
12         continue;
13     counts[prefix] ← counts[prefix] + 1;
14     opId ← MakeOperationId(prefix, counts[prefix]);
15     W[opId] ← BuildWritePattern(op.accesses);
16 return W;
```

After this step, the algorithm initializes the set W, responsible for storing the extracted *write patterns*, and the *counts* structure, which maintains independent

counters for each type of write operation: C for creation operations, U for update operations, and D for deletion operations. These counters generate incremental identifiers for the extracted patterns, such as C1, U1, and D1.

In the following lines, the algorithm processes each collected operation individually. If the analyzed operation does not represent a database write, it is discarded because it does not contribute to the write-cost calculation. For the remaining operations, the algorithm identifies the write type through the GetWriteKind function. The TypePrefix function then converts this type into a standardized prefix indicating whether the operation corresponds to a creation, update, or deletion. If the prefix cannot be determined, the operation is also ignored.

Once the write type is recognized, the corresponding counter is incremented, and a unique identifier is generated by the MakeOperationId function. Finally, the BuildWritePattern function transforms the accesses associated with the operation into a *write pattern*. This pattern records, for each collection involved, the number of auxiliary reads and writes performed. Ultimately, the final set W, returned at the end of the algorithm, gives the Evaluator module a concise representation of the write operations needed to estimate the maintenance cost of candidate schemas.

$$U1 = \{(\texttt{products}, 1, 1)\}$$

As an example, the product price update shown in Figure 2 can be represented by pattern $U1$. The prefix U denotes an update operation, whereas the number 1 indicates the sequential order in which this pattern was identified among the updates extracted from the DOS artifact. The pattern contains only the `products` collection because the operation acts directly on documents in this collection, locating the product and updating its price without accessing other collections. To perform this update, one auxiliary read is needed to locate the document, followed by one write to implement the modification.

3.3 Configuration File

In addition to artifacts extracted from the source code, EASE-Doc uses a configuration file to provide parameters that static analysis cannot obtain, such as collection volumes and the expected workload distribution. While the Extractor identifies collections, operations, and access patterns, the configuration file offers the quantitative context needed to assess candidate schemas. In Figure 6, the `MARKETPLACE` scenario considers the `products`, `sellers`, and `categories` collections. The `volume` section specifies the estimated number of documents in each collection, which the developer defines based on expected application usage.

The cost model then uses these estimates to evaluate the impact of read and write operations under different schema alternatives. This is because collection volume directly affects factors such as the potential number of documents accessed, the size of the manipulated structures, the cost of data retrieval, and the effort required to maintain duplicated attributes during update operations [1].

The `workload` section defines the workload composition by operation frequency and specifies how the cost model weights operations extracted from the source code.

```
{
  "scenario_id": "MARKETPLACE_PRODUCT",
  "database": {
    "name": "marketplace",
    "target": "MongoDB"
  },
  "volume": {
    "collections": {
      "products": 5000,
      "sellers": 200,
      "categories": 100
    }
  },
  "workload": {
    "alpha": 0.80,
    "read_mix": {
      "q1": 1.0
    },
    "write_mix": {
      "u1": 1.0
    }
  }
}
```

Fig. 6: Example of a configuration file for the *Marketplace* scenario.

The parameter `alpha = 0.80` represents the overall proportion of the workload associated with read operations relative to write operations. In this scenario, queries account for 80% of the total workload, while write operations represent the remaining 20%. This parameter is not calculated from the sum of the values in `read_mix` and `write_mix`; instead, it explicitly defines the global read/write balance the user expects for the schema. Section 3.6 explains this parameter in greater detail.

The `read_mix` and `write_mix` objects define the internal distribution of the read and write operations extracted from the DOS model. Each set is normalized independently so its weights sum to 1. In this way, `read_mix` allocates the overall read weight across the queries, while `write_mix` allocates the write weight among the corresponding operations.

The association with DOS is established through the identifiers assigned to the extracted operations. In the example, `q1` represents the query that accesses `products`, `sellers`, and `categories`, while `u1` corresponds to the update of the `price` field in `products`. These objects do not define new operations; instead, they assign relative weights to the identified patterns, which the Evaluator module then uses to compute the weighted cost of candidate schemas.

The `read_mix` and `write_mix` objects further specify the internal distribution of the read and write operations identified from the DOS model. To standardize the configuration, we normalize each set independently: the sum of the weights in `read_mix` must equal 1, as must the sum of the weights in `write_mix`. Accordingly, the `read_mix` weights distribute the global read frequency among the queries, while the `write_mix` weights distribute the global write frequency among the write operations. Therefore, when only one operation exists in either set, its internal weight must be 1.0.

The configuration file and the DOS model are linked through the identifiers assigned to the extracted operations. DOS organizes this information into elements that represent database operations (*Database Parameter*) and storage structures (*Container*). In the example, the `q1` key in `read_mix` identifies the query that retrieves a product from `products` and, from that result, accesses `sellers` and `categories`. Similarly, `u1` in `write_mix` corresponds to the `updateOne()` operation responsible for updating the `price` field in `products`. The identifiers in the configuration file enable you to assign weights to the logical patterns extracted from DOS.

These weights can be defined based on the observed frequency of each access pattern. When the application is already running, they can be estimated over a representative observation window using access logs, monitoring tools, database *profilers*, or persistence-layer metrics. Let $f(q_i)$ denote the observed frequency of query q_i, and $f(w_j)$ the observed frequency of write operation w_j. The parameters can then be calculated as:

$$\alpha = \frac{\sum_i f(q_i)}{\sum_i f(q_i) + \sum_j f(w_j)}$$

$$\texttt{read_mix}(q_i) = \frac{f(q_i)}{\sum_k f(q_k)} \qquad \text{and} \qquad \texttt{write_mix}(w_j) = \frac{f(w_j)}{\sum_k f(w_k)}$$

Here, i identifies the specific read pattern whose weight is being calculated, j identifies the specific write pattern, and k is the summation index over all operations of the corresponding type. Therefore, $\sum_k f(q_k)$ represents the total observed frequency of all read patterns, whereas $\sum_k f(w_k)$ represents the total observed frequency of all write patterns.

The counting process should consider invocations of the logical patterns defined in the configuration file rather than each internal access that composes a query. For greenfield projects where no historical execution data are available, estimate these values from functional requirements, major use cases, expected system usage, and synthetic workloads that represent plausible scenarios. In this context, define workload profiles such as read-oriented, balanced, and write-oriented workloads, and perform a sensitivity analysis to determine whether variations in α, `read_mix`, and `write_mix` affect the selected schema. After deployment, these estimates can be refined using actual application measurements.

3.4 Selector Module

The Selector module, illustrated in Figure 1(b), is responsible for the decision-making stage of EASE-Doc. Based on the schema inferred by the Extractor module and the identified duplication opportunities, this module explores different refactoring alternatives. It uses the Evaluator module as a cost function to compare candidate schemas. Therefore, the Selector acts as the optimization framework, searching for a structural configuration that reduces the estimated cost for the specified workload profile. Rather than analyzing each duplication independently, it considers the combined effect of refactorings on reads, writes, and document sizes.

3.5 Representation of Candidate Schemas

Each candidate schema is derived from the base schema inferred by the Extractor module and represented by a set of binary flags associated with the identified duplication opportunities. Let $D = \{D_1, D_2, \ldots, D_n\}$ be the set of candidate duplications. A candidate schema s can be described by a binary configuration $f = \{d_1, d_2, \ldots, d_n\}$, where each $d_i \in \{0, 1\}$ indicates whether duplication D_i is enabled or disabled in the evaluated schema. In this context, $d_i = 1$ signifies that the i-th duplication has been applied, while $d_i = 0$ signifies that it has not been applied.

In the simplified *Marketplace* scenario, the Extractor module identified 2 duplication plans, represented in Figure 5. Therefore, the search space consists of binary combinations of these duplications, yielding up to $2^2 = 4$ candidate schemas. The base schema, referred to here as $S3$, with no duplications enabled, is represented as:

$$S3 = \{0, 0\}.$$

A candidate schema such as $S3_D1$, in turn, represents a variation of $S3$ in which only duplication $D1$ is enabled:

$$S3_D1 = \{1, 0\}.$$

In this example, $D1$ copies the `storeName` and `rating` attributes from `Seller` to `Product`, whereas $D2$ copies the `name` attribute from `Category` to `Product`. Therefore, a schema like $S3_D1_D2$ indicates the simultaneous activation of both candidate duplications and is conveyed internally by the vector:

$$S3_D1_D2 = \{1, 1\}.$$

Similarly,

$$S3_D2 = \{0, 1\}.$$

Therefore, the schema name summarizes the active duplications, while the binary vector explicitly represents its internal configuration during the search process. This representation also defines the neighborhood the Selector explores. From a current schema, each neighbor is obtained by flipping a single position in the vector, that is, by turning one duplication on or off. For example, from $S3_D1$, the neighbor $S3$ can be obtained by disabling $D1$, whereas $S3_D1_D2$ can be obtained by enabling $D2$. For each newly generated candidate, the Evaluator module recalculates the read and write patterns and the average document size, accounting for the effects of the active duplications on the total schema cost.

Candidate schemas are therefore not treated as independent structures, but as configurations derived from the same base schema. This representation allows the Selector to systematically explore different combinations of refactorings and compare their impact on the estimated cost while preserving traceability between each evaluated alternative and the duplications from which it was derived.

3.5.1 Candidate Schema Selection

The candidate schema space is explored using the *Tabu Search* algorithm [21], applied to the combinations of duplications identified by the Extractor module, as shown in Figure 5. This choice is motivated by the combinatorial nature of the schema selection problem, since each candidate duplication can be enabled or disabled, resulting in different combinations of refactorings. Simple greedy strategies, such as *hill climbing*, tend to select only moves that provide immediate improvements, which may lead to local optima and limit the exploration of potentially better alternatives [22]. In contrast, *Tabu Search* uses short-term memory, represented by the tabu list, to prevent cycles and avoid immediate returns to previously visited solutions. It also allows moves that do not immediately reduce the cost, thereby broadening the exploration of the search space [21, 22].

Algorithm 3 describes the schema selection process based on *Tabu Search*. The search receives as input an initial schema s_0, the configuration file I, the set of candidate duplications D, the maximum tabu-list size T, the maximum number of iterations M, and the limit of consecutive iterations without improvement P. The objective is to return the best schema s^* found during the exploration of the search space.

Initially, the algorithm sets both the current schema s and the best-known schema s^* to the initial schema s_0, in lines 1 and 2. The cost of the best schema is then computed by the EvaluateSchema function using configuration I in line 3. The tabu list L, which stores recently performed moves, is initialized as empty in line 4. In turn, counter p, used to track the number of iterations without improvement, is initialized in line 5.

The main search is performed between lines 6 and 34 and is limited to M iterations. At each iteration, the algorithm generates the neighborhood N of the current schema using the GenerateNeighbors function (line 7). Each neighbor represents an alternative obtained by turning one candidate duplication on or off in D. In lines 8–10, the variables used to store the best neighbor of the current iteration are initialized: schema s_b, the move m_b that generated it, and its cost c_b.

Between lines 11 and 20, each neighbor (s', m) is evaluated according to its cost c'. The algorithm checks whether move m belongs to the tabu list and, if so, applies the aspiration criterion, which allows the move to be accepted when it produces a solution better than the global best schema s^*. Tabu moves that do not satisfy this criterion are discarded, and the algorithm selects the lowest-cost neighbor among the admissible candidates.

After evaluating the neighborhood, if no admissible neighbor exists ($s_b = \varnothing$), the search terminates. Otherwise, s_b becomes the new current schema and the corresponding move is inserted into the tabu list. If the list exceeds T, remove the oldest move, retaining only the recent search history.

Algorithm 3: Schema selection based on *Tabu Search*

Input : initial schema s_0, configuration I, duplications D, tabu-list size T, maximum number of iterations M, no-improvement limit P
Output: best schema found s^*

$s \leftarrow s_0$;
$s^* \leftarrow s_0$;
$c^* \leftarrow$ EvaluateSchema(s^*, I);
$L \leftarrow \emptyset$; // tabu list
$p \leftarrow 0$; // iterations without improvement
for $i \leftarrow 1$ **to** M **do**
  $N \leftarrow$ GenerateNeighbors(s, D);
  $s_b \leftarrow \varnothing$;
  $m_b \leftarrow \varnothing$;
  $c_b \leftarrow \infty$;
  foreach $(s', m) \in N$ **do**
    $c' \leftarrow$ EvaluateSchema(s', I);
    $tabu \leftarrow m \in L$;
    $asp \leftarrow tabu$ **and** $c' < c^*$;
    if $tabu$ ***and not*** asp **then**
      continue;
    if $c' < c_b$ **then**
      $s_b \leftarrow s'$;
      $m_b \leftarrow m$;
      $c_b \leftarrow c'$;
  if $s_b = \varnothing$ **then**
    break;
  $s \leftarrow s_b$;
  $L \leftarrow L \cup \{m_b\}$;
  if $|L| > T$ **then**
    RemoveOldest(L);
  if $c_b < c^*$ **then**
    $s^* \leftarrow s_b$;
    $c^* \leftarrow c_b$;
    $p \leftarrow 0$;
  else
    $p \leftarrow p + 1$;
  if $p \geq P$ **then**
    break;
return s^*;

Finally, if the cost of the best neighbor c_b is lower than the global best cost c^*, both the best schema s^* and its cost are updated, and the counter of iterations without improvement is reset. Otherwise, this counter is incremented. The search terminates when the limit P of consecutive iterations without improvement is reached and ultimately returns the best schema found.

Unlike a simple greedy search, *Tabu Search* can explore temporarily worse solutions while preventing cycles through the tabu list, increasing its ability to escape local optima in spaces with multiple duplication combinations. The parameters T, M, and P can be determined through empirical calibration: T should be proportional to the neighborhood size to prevent immediate returns without excessively restricting exploration; M should allow enough iterations to reach different regions of the search space; and P should define the number of iterations without improvement after which the search is considered stagnant. In practice, these values can be adjusted through preliminary runs by considering both the stability of the best solution found and the processing time.

3.6 Evaluator Module

The Evaluator module assigns a cost to each candidate schema, allowing structural alternatives to be compared under the same usage scenario. In EASE-Doc, this total cost serves as a comparable measure across schemas under the same data volume, memory, and workload conditions, providing the basis for the Selector to decide which candidates to retain, discard, or further explore during the local search. In practice, in Algorithm 3, this step is implemented through the call EvaluateSchema(f', I), where I contains the parameters of the evaluation scenario and f' represents the candidate schema generated from the inferred schema. Formally, the objective function of this module, for a candidate schema s under an evaluation scenario w, is defined as:

$$\text{TotalCost}(s, w) = \alpha \cdot \text{ReadCost}(s, w) + (1 - \alpha) \cdot \text{WriteCost}(s, w) \tag{1}$$

where $\alpha \in [0, 1]$, provided in the configuration file shown in Figure 6, is the parameter that calibrates the model's sensitivity to the predominance of reads or writes in the *workload*. The read cost (ReadCost) and write cost (WriteCost) are estimated separately from the evaluated schema and the operation profile, considering the collection accesses required by each operation. These expressions capture the main trade-off modeled by the Evaluator: although duplication can reduce read cost by decreasing the number of collections a query traverses, it can also increase write cost by requiring additional updates to maintain consistency of replicated data. Formally, these costs are defined as:

$$\text{ReadCost}(s, w) = \sum_{q \in Q} w_q \cdot \sum_{(C,n) \in A(q,s)} n \cdot \text{Cost}_{rand}(C) \tag{2}$$

$$\text{WriteCost}(s, w) = \sum_{u \in U} w_u \cdot \sum_{C \in A(u,s)} \big(\text{reads}(u, C) + \text{writes}(u, C)\big) \cdot \text{Cost}_{rand}(C) \tag{3}$$

In Equation (2), Q denotes the set of read queries in the scenario, w_q corresponds to the relative weight of each query q, defined in `read_mix` in Figure 6, and $A(q, s)$ represents the set of accesses required by q under schema s, already accounting for the effect of enabled or disabled duplications. This information is essential for recovering the application's access pattern and can be directly obtained from the *DOS* model generated by the Extractor module. Each pair $(C, n) \in A(q, s)$ indicates that collection C is accessed n times during query execution. In Equation (3), U denotes the set of write operations in the scenario, also provided by the *DOS*; w_u is the relative weight of operation u, specified in the `write_mix` attribute of the configuration file shown in Figure 6; and $A(u, s)$ represents the set of collections accessed by that operation under schema s. In addition, reads(u, C) denotes the number of auxiliary reads performed on C, whereas writes(u, C) represents the number of writes actually performed on that collection.

To compute Equations (2) and (3), the module estimates, for each collection C, a unit random-access cost, denoted by $\text{Cost}_{rand}(C)$, which represents the physical cost of retrieving a document through the corresponding index and data blocks. Separating

index and data costs is necessary because random access involves both locating the document through the indexing structure and reading the blocks that store its contents [23]. This formulation is based on the model proposed by Hewasinghage et al. (2021) [12], from which the Cost_{rand} equation is derived. This cost is defined as the sum of two components associated with index and data access:

$$\text{Cost}_{rand}(C) = \text{Cost}_{idx}(C) + \text{Cost}_{data}(C) \tag{4}$$

In Equation (4), the components $\text{Cost}_{idx}(C)$ and $\text{Cost}_{data}(C)$ are weighted by the relative access costs of memory, T_m, and disk, T_d, according to the fractions of index and data blocks resident in memory. In the Evaluator module, some of these basic parameters are fixed in the implementation, including the memory allocated to data and indexes, the block size, and the access costs, following the approach of Hewasinghage et al. (2021) [12], which assumes cache and disk accesses in fixed-size blocks with constant cost per block. For clarity, we adopt $T_m = 0.01$ for memory and $T_d = 1.0$ for disk, illustrating the significant order-of-magnitude difference between these two storage levels [12, 24].

$$\text{Cost}_{idx}(C) = \frac{T_m \cdot P_i(C) + T_d \cdot (1 - P_i(C))}{2} \tag{5}$$

$$\text{Cost}_{data}(C) = \frac{T_m \cdot P_d(C) + T_d \cdot (1 - P_d(C))}{2} \tag{6}$$

In these expressions, $P_i(C)$ and $P_d(C)$ represent, respectively, the proportions of index and data blocks of collection C that remain in memory, and are calculated as:

$$P_i(C) = \frac{M_i(C)}{B_i(C)} \qquad \text{and} \qquad P_d(C) = \frac{M_d(C)}{B_d(C)} \tag{7}$$

with values bounded to the interval $[0, 1]$. Here, $M_i(C)$ and $M_d(C)$ denote the number of index and data blocks kept in memory for collection C, whereas $B_i(C)$ and $B_d(C)$ correspond to the total number of index and data blocks of that collection. The values of $M_i(C)$ and $M_d(C)$ result from the step that distributes the available memory among the collections of the evaluated schema, proportionally to the total size of their indexes and data, so that larger collections receive a greater share of memory. The values of $B_i(C)$ and $B_d(C)$, in turn, are calculated from the ratio between stored volume and block capacity, following the formulation of [12]:

$$B_i(C) = \left\lceil \frac{\text{Size}_i(C)}{\text{BlockSize}} \right\rceil \qquad \text{and} \qquad B_d(C) = \left\lceil \frac{\text{Size}_d(C)}{\text{BlockSize}} \right\rceil \tag{8}$$

where $\text{Size}_i(C)$ represents the estimated total index size of collection C, $\text{Size}_d(C)$ represents the total data size of collection C, and BlockSize denotes the block size adopted by the model. As discussed previously, some of the Evaluator's basic parameters are fixed in the implementation. Among them, BlockSize = 4 KB is adopted, a value consistent with the formulation of [12] and supported by the analysis presented in [25].

To obtain $\text{Size}_d(C)$ and $\text{Size}_i(C)$, the Evaluator first computes the average document size of each collection from the inferred schema and the enabled duplications, based on the structural information extracted into U-Schema and on predefined BSON

(*Binary JSON*) size specifications[1], the format used by MongoDB to represent documents. These estimates are then combined with the collection volumes the user provides in Figure 6 to build the collection statistics. For each collection, these statistics include aggregated information such as cardinality, average document size, total data volume, and index volume, which form the basis for calculating the data and index blocks the cost model considers.

In practice, the Evaluator derives these statistics from the logical schema structure, enabled duplications, and the scenario's volume parameters. It first recalculates the average document size, accounting for the additional fields and lists introduced by duplication. It then estimates the expected cardinality of each collection from the specified volume and projects the total amount of data to be stored.

4 Experimental Methodology

The experimental evaluation of EASE-Doc was structured to systematically analyze the quality of the framework's decisions and its ability to identify suitable schemas for different *workload* profiles. To this end, this section first presents the research questions guiding the experiments, then describes the case study used for validation and, finally, the experimental environment.

4.1 Research Questions

Following the *Goal-Question-Metric* (GQM) paradigm [26], we organize the experimental evaluation around a goal, decompose it into research questions, and then define metrics that quantify the aspects under investigation. In this work, we use this structure to guide the evaluation of EASE-Doc and to associate each research question with the experimental evidence needed to answer it.

RQ1: *How accurately does EASE-Doc rank alternative document schemas according to their actual performance?*

This question evaluates the EASE-Doc cost model's ability to distinguish and rank candidate schemas by expected performance. The goal is not to predict the exact execution time of each operation, but rather to determine whether the model adequately preserves the relative performance ordering among the evaluated alternatives.

RQ2: *How effectively and efficiently does EASE-Doc explore the schema design space compared with exhaustive search?*

This question investigates the Selector module's ability to reduce the search space without compromising solution quality. The *Tabu Search*-based strategy is compared with exhaustive search, which systematically evaluates all possible combinations in the solution space and therefore identifies the best existing solution according to the evaluated cost. The analysis considers the effort required to reach the same solution, determining whether EASE-Doc can direct the exploration toward promising regions while avoiding the evaluation of all alternatives.

[1] https://bsonspec.org/spec.html. Accessed: Jan. 12, 2026.

RQ3: *Do the schemas selected by EASE-Doc improve runtime performance under different read/write workload profiles?*

This analysis investigates whether the framework's duplication decisions yield performance gains when reads predominate and whether the model avoids disadvantageous redundancy in write-intensive scenarios.

4.2 Case Study

The evaluation adopts a MongoDB-based Marketplace application as a case study, representing a medium-sized e-commerce system. The application comprises 14 collections containing 89,050 documents, including 30,000 order items, 15,000 reviews, and 10,000 orders. Figure 7 presents the base schema inferred by Fernández-Candel et al. (2026) [11], including reference and nesting relationships among the collections. These relationships establish different access paths and opportunities for selective attribute duplication, enabling the evaluation of trade-offs between read performance and redundant-data maintenance costs. The framework implementation and case-study configuration are available as described in the Data Availability statement.

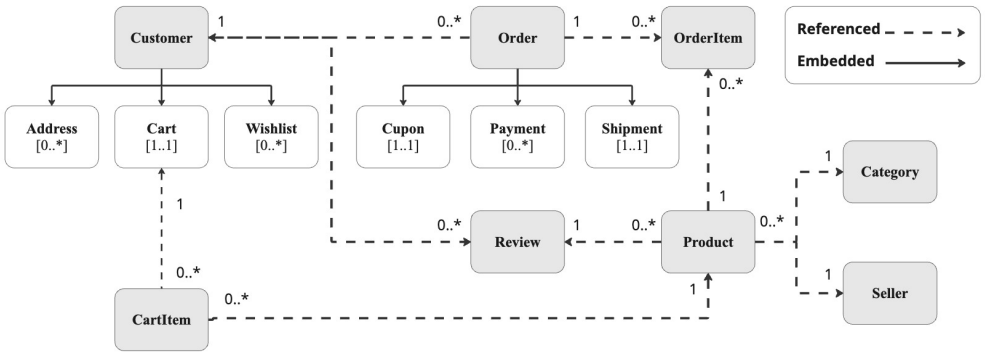


Fig. 7: Base schema of the case study.

The considered *workload* comprises 14 read operations and 7 write operations extracted from the application. The reads include queries involving orders, payments, shipments, products, carts, and reviews, ranging from single-collection accesses to operations that span multiple collections. Writes include updates to customers, products, sellers, categories, and orders, as well as creating orders and reviews. This combination lets us evaluate EASE-Doc under different read/write proportions, considering both the query benefits of duplication and the propagation cost of maintaining duplicated information. The source code and all operations used as input to EASE-Doc are publicly available[2].

The extraction process of Fernández-Candel et al. (2026) [11] also identified 14 duplication opportunities, presented in Figure 8. Each opportunity D_i represents a

[2]Case-study repository: https://github.com/morgan-oms/marketplace-easedoc

transformation that may or may not be applied to the base schema, involving the replication of attributes across related collections. Combining these decisions produces a set of $2^{14} = 16{,}384$ candidate schemas, which is subsequently explored by the EASE-Doc Selector module.

```
Duplication:
    D1.  Copy: Customer[name, email] to: Order
    D2.  Copy: Address[city, state] to: Order
    D3.  Copy: Coupon[code, discountValue] to: Order
    D4.  Copy: Product[name, price] to: OrderItem
    D5.  Copy: Seller[storeName] to: OrderItem
    D6.  Copy: Product[name, price] to: CartItem
    D7.  Copy: Product[name, price] to: Wishlist
    D8.  Copy: Seller[storeName, rating] to: Product
    D9.  Copy: Category[name] to: Product
    D10. Copy: Customer[name] to: Review
    D11. Copy: Product[name] to: Review
    D12. Copy: Order[orderDate, status] to: Payment
    D13. Copy: Order[orderDate, status] to: Shipment
    D14. Copy: Order[orderDate, status] to: OrderItem
```

Fig. 8: Duplication plan inferred for the EASE-Doc evaluation scenario.

4.3 Experimental Setup and Metrics

We conducted the experiments on MongoDB Atlas, using instances configured with MongoDB version 8.0.2. The experiments ran on a Dell G15 5520 laptop with an Intel Core i5-12500H processor at 2.50 GHz, 16 GB of RAM, a 500 GB SSD, and Windows 11 (64-bit). The *Tabu Search* algorithm used $T = 7$, $M = 80$, and $P = 20$, empirically defined for the search space of the *Marketplace* scenario [27, 28].

We selected a subset of candidate schemas for materialization in MongoDB and measured their performance using YCSB (Yahoo! Cloud Serving Benchmark), a benchmark designed to evaluate storage systems under different read and write workload profiles [29]. We chose YCSB because it lets us reproduce the workloads considered by EASE-Doc and compare estimated schema costs with empirical performance measurements. The *load* phase used `recordcount=89050`, while the *run* phase executed `operationcount=20000` operations with `threadcount=10`. We assessed performance using the average operation latency reported by YCSB, which we used to rank the evaluated schemas and compare the schemas selected by EASE-Doc with the base schema under different workload profiles.

To answer **RQ1**, Spearman's rank correlation coefficient (ρ) [30] is used, as it is suitable for measuring the association between two rankings without assuming a linear relationship between the original values. For each schema, we consider its rank according to the cost estimated by EASE-Doc and its corresponding rank according to the mean latency observed with YCSB. Since no ties occur among the evaluated values, the coefficient is calculated as:

$$\rho = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)}, \tag{9}$$

where d_i represents the difference between the ranks assigned to the i-th schema in the two rankings, and n corresponds to the number of evaluated schemas. Values close to 1 indicate stronger agreement between the rankings.

For **RQ2**, the efficiency of *Tabu Search* is compared with exhaustive search, considering that both select the same schema within the evaluated solution space. The analysis therefore focuses on the computational effort required to reach this solution, as recommended in experimental evaluations of heuristic methods [31, 32]. The number of evaluated schemas (N_{eval}), the search-space reduction, and the *speedup* are considered.

The search-space reduction is defined in this work as the proportion of alternatives that do not need to be evaluated relative to complete enumeration:

$$Reduction = \left(1 - \frac{N_{eval}}{N_{total}}\right) \times 100, \tag{10}$$

Finally, temporal efficiency is represented by *speedup*, a measure used to express the performance gain between computational strategies [33]:

$$Speedup = \frac{T_{exhaustive}}{T_{tabu}}, \tag{11}$$

where $T_{exhaustive}$ and T_{tabu} correspond to the execution times of exhaustive search and *Tabu Search*, respectively.

To answer **RQ3**, the schema selected by EASE-Doc is compared with the base schema generated by Fernández-Candel et al. (2026) [11], based on the weighted mean latency obtained with YCSB. We express the benefit as the percentage reduction in latency relative to the *baseline*, a common approach in performance evaluations to quantify relative gains between configurations [34, 35]. The metric is defined as:

$$Improvement = \frac{L_{base} - L_{EASE}}{L_{base}} \times 100, \tag{12}$$

where L_{base} represents the latency of the base schema and L_{EASE} the latency of the schema selected by EASE-Doc. The metric is calculated separately for each *workload* profile, allowing the performance gain to be quantified under different read/write proportions.

5 Results

This section presents the results of the experimental evaluation of EASE-Doc, organized by the research questions defined earlier. First, we analyze the cost model's ability to preserve the performance ordering of candidate schemas. Next, we evaluate the efficiency of *Tabu Search* in exploring the solution space compared with exhaustive search. Finally, we assess the practical impact of the selected schemas using measurements obtained with YCSB under different *workload* profiles.

5.1 Accuracy of Cost-based Ranking

Because the cost model guides selection among candidate schemas, we must verify whether its estimates adequately reflect observed performance. To this end, all

16,384 alternatives were ranked according to their estimated cost, and nine schemas distributed across the upper, middle, and lower regions of the ranking were materialized in MongoDB and evaluated with YCSB. We compared the two extreme profiles, R90_W10 and R10_W90, representing predominantly read-oriented and write-oriented scenarios, respectively.

Figures 9 and 10 compare EASE-Doc's estimated cost with the experimental latency measured using YCSB. Because these metrics have different units and scales, we express each value as a proportion relative to the best result in its series, which we set to 1. Thus, the comparison is not meant to assess numerical equivalence between cost and latency, but to determine whether the model's predicted relative degradation matches the experimentally observed behavior.

For R90_W10, Figure 9 shows that both curves exhibit a similar progression across the evaluated positions. The three highest-ranked schemas remain close to the best result, whereas the intermediate alternatives and, particularly, those at the lower end of the ranking show a progressive increase in both estimated cost and experimental latency. Although their relative magnitudes differ, no inversions appear in the ordering of the nine schemas. This behavior indicates that the cost model captures the relative benefit of structures favored by a predominantly read-oriented *workload*.

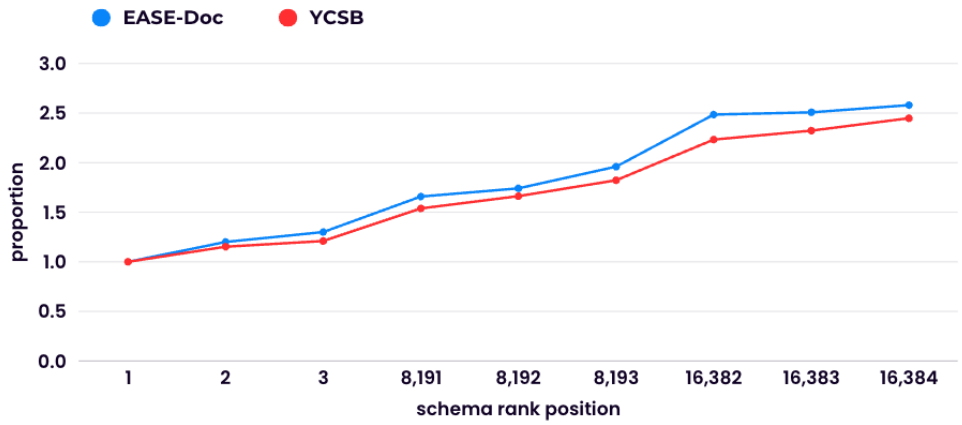


Fig. 9: Comparison between the relative cost estimated by EASE-Doc and the relative latency observed with YCSB under the *workload* **R90_W10**.

In the R10_W90 scenario, as shown in Figure 10, a more pronounced separation can be observed among groups of schemas, particularly in the intermediate and lower regions of the ranking. Nevertheless, the curves remain consistent: as the cost estimated by EASE-Doc increases, the latency observed with YCSB also increases. This result is consistent with the stronger influence of write operations in this profile, since structures with greater redundancy incur additional costs from update propagation.

The visually observed agreement was quantified using Spearman's rank correlation coefficient (ρ) [30]. For each *workload* profile, the nine pairs formed by the cost

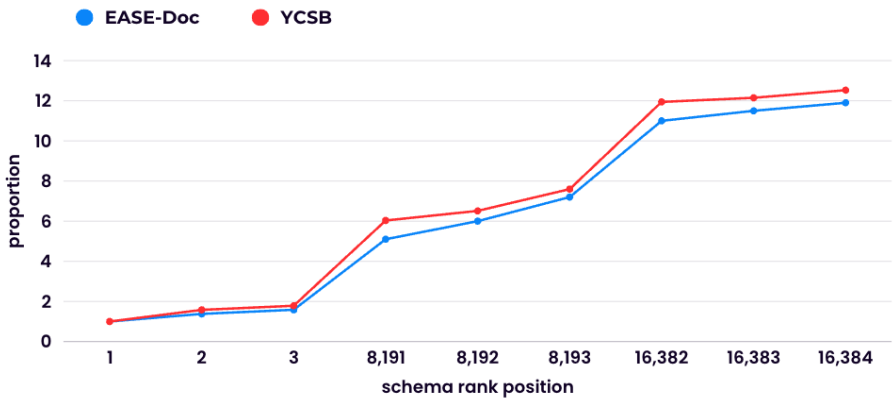


Fig. 10: Comparison between the relative cost estimated by EASE-Doc and the relative latency observed with YCSB under the *workload* **R10_W90**.

produced by the EASE-Doc model and the average latency obtained with YCSB for the same schema were considered. These values were converted into relative ranks and compared using the Spearman formula presented in Equation 9. The results are shown in Table 2.

Table 2: Correlation between the cost model and the results obtained with YCSB.

Workload	Schemas	ρ
R90_W10	9	1.0
R10_W90	9	1.0

For both profiles, $\rho = 1.0$ indicates perfect agreement in the ranks of the nine schemas between the cost estimated by EASE-Doc and the average latency observed with YCSB. Although the two metrics differ in magnitude, we observed no inversions in the ordering of the evaluated schemas.

Therefore, the results address **RQ1** by demonstrating that the EASE-Doc cost model provides a ranking that aligns with experimentally observed performance across the two extreme *workload* profiles. This property is particularly relevant to EASE-Doc because the model does not directly predict latency in time units, but instead provides a comparative measure that distinguishes more suitable from less suitable alternatives. The observed agreement therefore supports using estimated cost as the evaluation function for candidate schema selection.

5.2 Search Effectiveness and Efficiency

After verifying the cost model's ability to rank candidate schemas, the second analysis focuses on the efficiency of the selection process. For this purpose, the *Tabu Search* employed by EASE-Doc is compared with exhaustive search, which evaluates all $2^{14} = 16{,}384$ possible combinations and therefore provides a reference for the best solution within the considered search space. We compared performance under three *workload* profiles: R90_W10, R60_W40, and R10_W90.

In all scenarios, *Tabu Search* identified the same lowest-cost schema found by exhaustive search while evaluating only a fraction of the available alternatives. Table 3 presents the number of schemas evaluated by *Tabu Search* (N_{eval}), the search-space reduction calculated according to Equation 10, the execution times of both strategies, and the *speedup* obtained using Equation 11.

Table 3: Effectiveness and efficiency of *Tabu Search* compared with exhaustive search.

Workload	N_{eval}	N_{total}	**Reduction**	$T_{exhaustive}$ **(s)**	T_{tabu} **(s)**	**Speedup**
R90_W10	203	16,384	98.76%	0.75	0.012363	60.66×
R60_W40	192	16,384	98.83%	0.75	0.012363	60.66×
R10_W90	192	16,384	98.83%	0.75	0.011365	65.99×

In the R90_W10 scenario, EASE-Doc evaluated 203 out of the 16,384 candidate schemas, corresponding to a 98.76% reduction in the search space. Under the R60_W40 and R10_W90 profiles, we evaluated 192 alternatives, reducing the search space by 98.83%. In all cases, we explored less than 1.3% of the total search space and still found the same solution as complete enumeration.

Although exhaustive search completed in approximately 0.75 s, Tabu Search's main advantage is reducing the number of evaluated schemas while preserving solution quality. Across all workload profiles, the heuristic identified the same minimum-cost schemas as exhaustive search while evaluating less than 1.3% of the candidate space.

This reduction becomes particularly relevant when evaluating candidates requires executing queries against materialized schemas rather than relying solely on analytical cost estimates. In such cases, exhaustive evaluation may incur substantial overhead, further aggravated by the exponential growth of the design space. Although we did not experimentally evaluate these scenarios, the results answer **RQ2** by showing that Tabu Search substantially reduces search effort while identifying the same minimum-cost solutions as exhaustive search across the investigated workloads.

5.3 Runtime Performance of Selected Schemas

The previous analysis demonstrated that EASE-Doc can substantially reduce the search space without changing the solution identified by exhaustive enumeration. This analysis now focuses on the actual performance of the selected schemas. To answer **RQ3**, we compared each EASE-Doc-selected solution with the base schema $S3$, derived from the structure inferred by Fernández-Candel et al. (2026) [11] before applying the

framework's proposed duplications. The comparison uses the weighted-average latency measured with YCSB and the relative improvement calculated using Equation 12.

Table 4 presents the results for the three *workload* profiles, including the weighted average latency of the base schema (L_{base}), the latency of the schema selected by EASE-Doc (L_{EASE}), and the relative improvement between them. The selected set of duplications varies with the read/write proportion, reflecting the trade-off between reducing additional cross-collection accesses and limiting the maintenance cost of redundant information.

Table 4: Performance of the schemas selected by EASE-Doc compared with the base schema S3.

Workload	**Selected schema**	L_{base}	L_{EASE}	**Improv.**
R90_W10	S3_D2_D3_D6_D7_D11_D12_D13_D14	6,209,309.69	5,348,939.57	13.86%
R60_W40	S3_D2_D3	7,654,309.57	5,763,695.11	24.70%
R10_W90	S3	10,062,642.71	10,062,642.71	0.00%

Under the predominantly read-oriented R90_W10 profile, EASE-Doc selected eight duplication opportunities, resulting in the schema S3_D2_D3_D6_D7_D11_D12_D13_D14. The weighted average latency decreased from 6,209,309.69 μs to 5,348,939.57 μs, corresponding to an improvement of 13.86%. In this scenario, the larger proportion of read operations favors selectively duplicating information used by queries that, under the base schema, would require additional accesses to other collections.

The largest improvement was observed under the intermediate R60_W40 profile. In this case, only duplications D2 and D3 were selected, reducing latency from 7,654,309.57 μs to 5,763,695.11 μs, corresponding to an improvement of 24.70%. This result shows that the benefit does not depend on the number of applied duplications, but rather on their suitability for the *workload*: a smaller set of refactorings was sufficient to reduce read costs without introducing an excessive penalty on write operations.

Under R10_W90, in contrast, the base schema $S3$ itself was selected as the lowest-cost alternative. Consequently, latency remained at 10,062,642.71 µs, as the framework preserved the base schema without introducing additional duplication costs. This behavior is consistent with the predominance of writes, since redundancy would increase the cost of propagating updates while reducing the benefit of duplication.

The results answer **RQ3** by showing that EASE-Doc adapts the selected structure to the *workload* characteristics. In scenarios with a higher proportion of reads, the framework introduced selective duplications and achieved latency reductions of up to 24.70%; when writes became predominant, it preserved the base schema.

6 Conclusion

This paper presented EASE-Doc, a workload-aware framework for extracting, evaluating, and selecting schemas for document-oriented NoSQL databases. The framework integrates code-based reverse engineering with the derivation of read and write patterns, cost-based evaluation, and heuristic exploration of selective duplication alternatives. By explicitly considering the propagation cost of updates, EASE-Doc evaluates whether the read benefits introduced by data duplication compensate for the additional maintenance required under a given workload.

The experimental results provide evidence for the three aspects investigated in this study. First, the cost model preserved the performance ordering observed with YCSB for the nine schemas evaluated under both extreme workload profiles, resulting in a Spearman correlation of $\rho = 1.0$. Second, Tabu Search identified the same minimum-cost schemas as exhaustive search across all workload profiles while evaluating less than 1.3% of the 16,384 candidates. This substantial reduction in search effort is particularly relevant for larger design spaces or scenarios involving more expensive schema evaluations. Finally, the schemas selected by EASE-Doc adapted to the read-/write composition of the workload: selective duplications reduced weighted average latency by 13.86% in R90_W10 and 24.70% in R60_W40, whereas the base schema was retained in R10_W90. These results indicate that the number of applied duplications is less relevant than selecting those whose benefits outweigh their write-maintenance costs.

Future work will therefore extend the extraction process to additional programming ecosystems, such as Java and Python, and evaluate the framework on other document database systems and application domains. Further experiments with larger data volumes and more heterogeneous workloads are also needed to assess the generality and scalability of the approach. Finally, the cost model can be refined to incorporate additional execution factors and improve its representation of production environments. Overall, the results obtained in this study support EASE-Doc as a promising decision-support approach for making document-schema refactoring more systematic and explicitly driven by the trade-off between read performance and write-maintenance cost.

Statements and Declarations

Funding

The authors received no specific funding for this research.

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Data Availability

The EASE-Doc framework, the MongoDB Marketplace case study, and the experimental configurations and results supporting this study are publicly available at

https://github.com/morgan-oms/ease-doc and https://github.com/morgan-oms/marketplace-easedoc.

References

[1] Sadalage, P.J., Fowler, M.: NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, p. 192. Addison-Wesley, Upper Saddle River, NJ (2012)

[2] Fernandez-Candel, C., Sevilla, D., García-Molina, J.: A unified metamodel for nosql and relational databases. Information Systems **104**, 101898 (2021) https://doi.org/10.1016/j.is.2021.101898

[3] Gómez, P., Casallas, R., Roncancio, C.: Data schema does matter, even in NoSQL systems! In: 2016 IEEE Tenth International Conference on Research Challenges in Information Science (RCIS), Grenoble, France (2016). https://doi.org/10.1109/RCIS.2016.7549340 . https://hal.science/hal-01482250

[4] Gómez, P., Roncancio, C., Casallas, R.: Analysis and evaluation of document-oriented structures. Data & Knowledge Engineering **134**, 101893 (2021) https://doi.org/10.1016/j.datak.2021.101893

[5] Lima, C., Santos Mello, R.: A workload-driven logical design approach for nosql document databases. In: Proceedings of the 17th International Conference on Information Integration and Web-Based Applications & Services. iiWAS '15. Association for Computing Machinery, New York, NY, USA (2015). https://doi.org/10.1145/2837185.2837218 . https://doi.org/10.1145/2837185.2837218

[6] Chen, L., Davoudian, A., Liu, M.: A workload-driven method for designing aggregate-oriented nosql databases. Data & Knowledge Engineering **142**, 102089 (2022) https://doi.org/10.1016/j.datak.2022.102089

[7] Hewasinghage, M., Nadal, S., Abelló, A., Zimányi, E.: Automated database design for document stores with multicriteria optimization. Knowl. Inf. Syst. **65**(7), 3045–3078 (2023) https://doi.org/10.1007/s10115-023-01828-3

[8] Abubakar Imam, A., Basri, S., Ahmad, R., Watada, J., González-Aparicio, M., Almomani, M.A.: Data modeling guidelines for nosql document-store databases. International Journal of Advanced Computer Science and Applications **9** (2018) https://doi.org/10.14569/IJACSA.2018.091066

[9] Klettke, M., Störl, U., Scherzinger, S.: Schema extraction and structural outlier detection for json-based nosql data stores, 425–444 (2015)

[10] Frozza, A.A., Santos Mello, R., Costa, F.d.S.: An approach for schema extraction of json and extended json document collections. In: Proceedings of the 2018 IEEE International Conference on Information Reuse and Integration (IRI), pp. 356–363. IEEE, Salt Lake City, UT, USA (2018). https://doi.org/10.1109/IRI.2018.0

0060 . https://ieee.org

[11] Fernandez-Candel, C.J., Cleve, A., Garcia-Molina, J.J.: Towards the automated extraction and refactoring of nosql schemas from application code. Journal of Systems and Software **236**, 112787 (2026) https://doi.org/10.1016/j.jss.2026.112787

[12] Hewasinghage, M., Abelló, A., Varga, J., Zimányi, E.: A cost model for random access queries in document stores. The VLDB Journal **30**(4), 559–578 (2021) https://doi.org/10.1007/s00778-021-00660-x

[13] MongoDB: Building with Patterns: The Extended Reference Pattern. https://www.mongodb.com/company/blog/building-with-patterns-the-extended-reference-pattern. Acesso em: 24 de maio de 2026 (2019)

[14] Min, M.: Modeling with design patterns in mongodb for public transportation data. The International Journal of Internet, Broadcasting and Communication **16**(4), 460–465 (2024) https://doi.org/10.7236/IJIBC.2024.16.4.460

[15] Abdelhédi, F., Rajhi, A., Zouari, G., Akoka, J., Feki, J., Wattiau, I.C.: Extraction process of the logical schema of a document-oriented nosql database. In: Proceedings of the 10th International Conference on Model-Driven Engineering and Software Development (MODELSWARD 2022), pp. 61–71 (2022). https://doi.org/10.5220/0010899000003119 . SciTePress. https://www.scitepress.org/Papers/2022/108990/

[16] Koupil, P., Hricko, S., Holubová, I.: A universal approach for multi-model schema inference. Journal of Big Data **9**(97) (2022) https://doi.org/10.1186/s40537-022-00645-9

[17] Brambilla, M., Cabot, J., Wimmer, M.: Model-Driven Software Engineering in Practice, 2nd edn. Synthesis Lectures on Software Engineering. Morgan & Claypool Publishers, San Rafael, CA (2017). https://acm.org

[18] Mior, M.J., Salem, K., Aboulnaga, A., Liu, R.: Nose: Schema design for nosql applications. IEEE Transactions on Knowledge and Data Engineering **29**, 2275–2289 (2016)

[19] Vera-Olivera, H., Alvarez-Mamani, E., Holanda, M.: Análise de desempenho em banco de dados nosql orientado a documentos: Um Índice para comparação de modelos de dados. In: Anais do XXXVIII Simpósio Brasileiro de Bancos de Dados, pp. 26–38. SBC, Porto Alegre, RS, Brasil (2023). https://doi.org/10.5753/sbbd.2023.231721 . https://sol.sbc.org.br/index.php/sbbd/article/view/25514

[20] Hewasinghage, M., Abelló, A., Varga, J., Zimányi, E.: Docdesign: Cost-based database design for document stores. In: Proceedings of the 32nd International Conference on Scientific and Statistical Database Management. SSDBM '20.

Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3400903.3401689 . https://doi.org/10.1145/3400903.3401689

[21] Burke, E., Kendall, G.: Search Methodologies—Introductory Tutorials in Optimization and Decision Support Techniques, (2005). https://doi.org/10.1007/0-387-28356-0

[22] Gendreau, M., Potvin, J.-Y. (eds.): Handbook of Metaheuristics, 3rd edn. International Series in Operations Research & Management Science, vol. 272. Springer, Cham, Switzerland (2019). https://doi.org/10.1007/978-3-319-91086-4

[23] Elmasri, R., Navathe, S.B.: Fundamentals of Database Systems, 7th edn. Pearson, Boston, MA (2015)

[24] Ousterhout, J., Agrawal, P., Erickson, D., Kozyrakis, C., Leverich, J., Mazières, D., Mitra, S., Narayanan, A., Parulkar, G., Rosenblum, M., Rumble, S., Stratmann, E., Stutsman, R.: The case for ramclouds: Scalable high-performance storage entirely in dram. Operating Systems Review **43**, 92–105 (2009) https://doi.org/10.1145/1713254.1713276

[25] Petrov, I., Gottstein, R., Ivanov, T., Bausch, D., Buchmann, A.: Page size selection for oltp databases on ssd storage. JIDM **2**, 11–18 (2011)

[26] Basili, V.R., Caldiera, G., Rombach, H.D.: In: Marciniak, J.J. (ed.) The Goal Question Metric Approach, vol. 1, pp. 528–532. John Wiley & Sons, New York, NY (1994)

[27] Boschetti, M.A., et al.: Contemporary approaches in matheuristics: An updated survey. Annals of Operations Research (2024)

[28] Konovalenko, A., *et al.*: Exploring tabu tenure policies with machine learning. Electronics **14**(13), 2642 (2025)

[29] Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R.: Benchmarking cloud serving systems with ycsb. In: Proceedings of the 1st ACM Symposium on Cloud Computing. SoCC '10, pp. 143–154. Association for Computing Machinery, New York, NY, USA (2010). https://doi.org/10.1145/1807128.1807152

[30] Spearman, C.: The proof and measurement of association between two things. The American Journal of Psychology **15**(1), 72–101 (1904) https://doi.org/10.2307/1412159

[31] Rardin, R.L., Uzsoy, R.: Experimental evaluation of heuristic optimization algorithms: A tutorial. Journal of Heuristics **7**, 261–304 (2001) https://doi.org/10.1023/A:1011319115230

[32] Barr, R.S., Golden, B.L., Kelly, J.P., Resende, M.G.C., Stewart, W.R.: Designing and reporting on computational experiments with heuristic methods. Journal of Heuristics **1**, 9–32 (1995) https://doi.org/10.1007/BF02430363

[33] Barr, R.S., Hickman, B.L.: Reporting computational experiments with parallel algorithms: Issues, measures, and experts' opinions. ORSA Journal on Computing **5**(1), 2–18 (1993) https://doi.org/10.1287/ijoc.5.1.2

[34] Jain, R.: The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling. Wiley, New York (1991)

[35] Fleming, P.J., Wallace, J.J.: How not to lie with statistics: The correct way to summarize benchmark results. Communications of the ACM **29**(3), 218–221 (1986) https://doi.org/10.1145/5666.5673